

hash_trackerv2 — Study Guide

A concept-by-concept learning path for understanding the piracy demand-feed pipeline before reviewing the code. Each topic maps to the file it explains. Blue text is a clickable link. Generated 2026-06-01.

How to use this guide

Work top to bottom. By the end of **Tier 2** you'll understand the harvester; by **Tier 4** the DHT counter will make sense; **Tier 5** covers the country export; **Tier 6** covers the infrastructure and this week's fixes. The official spec pages (bittorrent.org BEPs), SQLite, MaxMind, AWS and Wikipedia links are stable. For videos I give exact search phrases plus a suggested channel, so you pick the clearest current one.

TIER 0 — Start here (~30 min, do these 3 first)

- 1 **Watch:** ■ [BitTorrent Explained — what it is & how it works](#) then ■ [Distributed Hash Tables — Computerphile](#).
- 2 **Read:** Wikipedia [BitTorrent](#) then [Distributed hash table](#) — for the vocabulary.
- 3 **Read:** [BEP 5 — DHT Protocol](#) — this is the protocol our biggest file implements.

TIER 1 — Torrent fundamentals

Concepts: infohash, magnet link, the swarm, seeders vs leechers vs peers — the atoms of all our data.

Maps to: **(underpins everything)**

- Wikipedia: [Magnet URI scheme](#), [Torrent file](#)
- [BEP 3](#) — the original BitTorrent spec
- [BEP 9](#) — magnet links & metadata exchange
- ■ [BitTorrent Explained \(beginner\)](#) · *search YouTube: “seeders leechers peers explained”*

TIER 2 — Trackers: announce & scrape

Concepts: asking a tracker “who is in this swarm?” (announce) and “how many?” (scrape); HTTP vs UDP trackers.

Maps to: **tracker_harvest.py, tracker_harvest_service.py (our heavy lifter – most real peer IPs)**

- [BEP 15](#) — UDP tracker protocol
- Wikipedia: [BitTorrent tracker](#)
- ■ [A Deep Dive Into the BitTorrent Protocol](#) — covers peers, trackers & announce in depth

TIER 3 — DHT & Kademlia (the core, the hard part)

Concepts: trackerless peer discovery, node IDs, XOR distance, the routing table, the iterative get_peers / find_node walk.

Maps to: **dht_peer_count.py (our largest, most important file)**

- [BEP 5](#) — read carefully this time
- Wikipedia: [Kademlia](#), [Mainline DHT](#)
- ■ [Kademlia, Explained](#) — best starting point
- ■ [Kademlia: A DHT Routing Protocol \(lecture\)](#) — deeper, academic

- Original paper: [search YouTube: “Kademlia XOR metric Maymounkov Mazieres PDF”](#) (intro section is readable)

TIER 4 — The BEP extensions we specifically use

Concepts: DHT seeder/leecher estimates via bloom filters (why we treat DHT counts as a WEAK sample); sampling infohashes from the DHT.

Maps to: [bep33 logic inside dht_peer_count.py](#), [bep51_crawler.py](#)

- [BEP 33](#) — DHT scrape (bloom-filter estimates)
- [BEP 51](#) — DHT infohash indexing / sampling

TIER 5 — IP → Country / ASN (the “demand by country” part)

Concepts: geolocating an IP to a country; what an ASN is; why we flag datacenter / VPN ranges.

Maps to: [export_nbcu.py](#), [export_asn_ab.py](#), the [GeoLite2 .mmdb files](#)

- [MaxMind GeoLite2 docs](#)
- Wikipedia: [Autonomous system \(Internet\)](#)
- ■ [What is an Autonomous System Number \(ASN\)?](#)

TIER 6 — Infrastructure & this week's fixes

Concepts: the layers you'll review in the unit files and the crash-loop / watchdog changes.

Maps to: [systemd unit files](#), [dht_peer_count.py \(SQLite fix\)](#), [health_watchdog.py](#)

- **SQLite WAL & locking** (the crash-loop root cause): [wal.html](#), [pragma.html](#) (read busy_timeout, wal_checkpoint, synchronous). ■ [SQLite WAL mode \(High Performance SQLite\)](#)
- **systemd** (our 8 workers + the start-limit trap): [systemd.service](#), [systemd.timer](#). ■ [systemd: Intro & Unit Files](#)
- **Python asyncio + UDP** (one worker = 30–50 concurrent DHT queries): ■ [Python AsyncIO Explained in 9 Minutes](#)
- **AWS we rely on:** EC2, SSM Session Manager (how commands run on the box), SNS (your alerts), S3, CloudWatch, IAM instance role. ■ [Connect to EC2 securely with SSM Session Manager](#)

Cheat sheet — which file each concept explains

File	What it does	Read first
tracker_harvest.py / _service.py	Harvests peer IPs from trackers (announce/scrape). The heavy lifter.	Tier 1–2
dht_peer_count.py	Walks the DHT (Kademlia get_peers) to count peers per infohash. 8 workers.	Tier 3–4
bep51_crawler.py	Samples new infohashes from the DHT (BEP 51).	Tier 4
trending_hash_collector.py / collect.py	Discovers which titles/hashtags to track (TMDB, trackers, AniList).	Tier 1
export_nbcu.py / export_asn_ab.py	Builds the daily distinct-peer-IP-per-country feed; ASN/datacenter analysis.	Tier 5

<code>health_watchdog.py</code>	Whole-pipeline health check → SNS (incl. the new midnight grace window).	Tier 6
<code>systemd unit files</code>	Run/keep-alive the workers, harvester, daily export timer.	Tier 6

One mental model that ties it together: We measure piracy demand by counting **distinct peer IPs per title, per country, per day**. Two independent sources feed it: (1) the **tracker harvester** — the bulk of real IPs, and (2) the **DHT counter** — a weaker, supplementary sample. We geolocate every IP to a country, drop noise, and write the daily CSV. NBCU's third-party feed is only our *benchmark* for sanity-checking — we built our own.

Tip: keep this open beside the repo's docs/11_END_TO_END_FLOW.md and docs/11_ops_runbook.md while you review.